

第 2 章

指令：計算機的語言

2.1 介紹

2.2 計算機硬體的運作

2.3 計算機硬體的運算元

2.4 有號及無號數字

2.5 在計算機中表示指令

2.6 邏輯運算

2.7 能作決定的指令

2.8 以計算機硬體來幫助程序呼叫

2.9 MIPS 對32 位元立即值及位址的定址法

2.10 平行性與指令：同步

2.11 翻譯與啟動一個程式

2.12 一個結合所有觀念的C 排序
程式例

2.13 進階教材：編譯C 語言

2.14 實例：ARMv7 (32 位元)指令

2.15 實例：x86 指令

2.16 實例：ARMv8 (64 位元)指令

2.17 謬誤與陷阱

2.18 總結評論

2.19 歷史觀點與進一步閱讀

2.1 介紹

- ▣ 計算機語言中的單字稱為指令(instructions), 而它的字彙稱為一個指令集(instructionset)
- ▣ 所選的指令集源自於MIPS Technologies 公司
- ▣ 內儲程式的觀念(stored-program concept), 圖2.1 為本章所涵蓋的指令集

↳ 將所有欲執行之程式放在 mem 中

圖2.1 本章中會出現的MIPS 組合語言指令³⁻¹

MIPS 運算元

名 稱	舉 例	註 解
32 個暫存器	\$s0-\$s7, \$t0-\$t9, \$zero, \$a0-\$a3, \$v0-\$v1, \$gp, \$fp, \$sp, \$ra, \$at	快速存取資料的地方。在 MIPS 中，資料必須要在暫存器裡才能執行運算，暫存器 \$zero 恆等於 0，而暫存器 \$at 則是保留給組譯器來處理值很大的常數。
2 ³⁰ 記憶體字組	Memory[0], Memory[4], ..., Memory[4294967292]	在 MIPS 裡，記憶體只能由資料傳輸指令來存取。MIPS 使用位元組位址，所以連續的字組位址相差 4。記憶體裡儲存資料結構、陣列和溢出暫存器 (spilled registers)。

4 個 byte 為 1 字組，一個 byte 8 個 bit

MIPS 組合語言

分類	指 令	舉 例	意 義	註 解
算術	加法	add \$s1,\$s2,\$s3	\$s1=\$s2+\$s3	三個暫存器運算元
	減法	sub \$s1,\$s2,\$s3	\$s1=\$s2-\$s3	三個暫存器運算元
	加立即值	addi \$s1,\$s2,20	\$s1=\$s2+20	加上常數

→ 算術、Logic 運算都只能對暫存器作運算，
運算結果只能存回暫存器。

圖2.1 本章中會出現的MIPS 組合語言指令³⁻²

MIPS 組合語言

分類	指 令	舉 例	意 義	註 解
資料 傳輸	載入字組	lw \$s1,20(\$s2)	\$s1=Memory[\$s2+20]	字組由記憶體載入至暫存器 <i>mem → reg</i>
	儲存字組	sw \$s1,20(\$s2)	Memory[\$s2+20]=\$s1	字組由暫存器儲存至記憶體 <i>reg → mem</i>
	載入半字組	lh \$s1,20(\$s2)	\$s1=Memory[\$s2+20]	半字組由記憶體載入至暫存器
	載入無號 半字組	lhu \$s1,20(\$s2)	\$s1=Memory[\$s2+20]	無號的半字組由記憶體載入至暫存器
	儲存半字組	sh \$s1,20(\$s2)	Memory[\$s2+20]=\$s1	半字組由暫存器儲存至記憶體
	載入位元組	lb \$s1,20(\$s2)	\$s1=Memory[\$s2+20]	位元組由記憶體載入至暫存器
	載入無號 位元組	lbu \$s1,20(\$s2)	\$s1=Memory[\$s2+20]	無號的位元組由記憶體載入至暫存器
	儲存位元組	sb \$s1,20(\$s2)	Memory[\$s2+20]=\$s1	位元組由暫存器儲存至記憶體
	載入連結的字元 組	ll \$s1,20(\$s2)	\$s1=Memory[\$s2+20]	作為不可分割的（記憶體與儲存器內容）交換中第一部份的載入字元組
	條件式儲存字元 組	sc \$s1,20(\$s2)	Memory[\$s2+20]=\$s1; \$s1=0 或 1	作為不可分割的（記憶體與暫存器內容）交換中第二部份的儲存字組
	載入上半部立即 值	lui \$s1,20	\$s1=20*2 ¹⁶	載入常數至較高的 16 位元
邏輯	及	and \$s1,\$s2,\$s3	\$s1=\$s2 & \$s3	三個暫存器運算元：逐位元的及運算
	或	or \$s1,\$s2,\$s3	\$s1=\$s2 \$s3	三個暫存器運算元：逐位元的或運算
	反或	nor \$s1,\$s2,\$s3	\$s1=~(\$s2 \$s3)	三個暫存器運算元：逐位元的反或運算
	及立即值	andi \$s1,\$s2,20 <i>(bin.)</i>	\$s1=\$s2 & 20	暫存器與常數做逐位元的及運算
	或立即值	ori \$s1,\$s2,20 <i>(bin.)</i>	\$s1=\$s2 20	暫存器與常數做逐位元的或運算
	邏輯左移	sll \$s1,\$s2,10	\$s1=\$s2<<10	左移常數個位元位置
	邏輯右移	srl \$s1,\$s2,10	\$s1=\$s2>>10	右移常數個位元位置

圖2.1 本章中會出現的MIPS 組合語言指令3-3

Program Counter 程式計數器
PC+4 : 現在執行指令的下一行

往下跳100個位置 MIPS 組合語言

分類	指 令	舉 例	意 義	註 解
條件式分支	若等於則分支 Branch Equal	beq \$s1,\$s2,25	若(\$s1==\$s2)則前往 PC+4+100 → 從下一行開始數100個位置	等於測試：PC 相對的分支
	若不等於則分支 Branch Not Equal	bne \$s1,\$s2,25	若(\$s1!=\$s2)則前往 PC+4+100	不等於測試：PC 相對的分支
	若小於則設定 Set less Than	slt \$s1,\$s2,\$s3	若(\$s2<\$s3) \$s1=1; 否則 \$s1=0	小於比較；用於 beq、bne
	無號若小於則設定	sltu \$s1,\$s2,\$s3	若(\$s2<\$s3) \$s1=1; 否則 \$s1=0	無號數的小於比較
	若小於立即值則設定	slti \$s1,\$s2,20	若(\$s2<20) \$s1=1; 否則 \$s1=0	小於某常數的比較
	無號若小於立即值則設定	sltiu \$s1,\$s2,20	若(\$s2<20) \$s1=1; 否則 \$s1=0	無號數的小於某常數的比較
非條件式跳躍	跳躍 (跳回主程式)	j 2500	前往 10000	跳至目的位址
	透過暫存器跳躍	jr \$ra	前往 \$ra	用於 switch 敘述、程序反回
	跳躍並連結 (跳到副程式)	jal 2500	\$ra=PC+4 前往 10000	用於程序呼叫

Important!

Handwritten notes:
 * 相對的 (往下跳25個指令)
 * 每個Cmd都是一word (4 byte)長!

2.2 計算機硬體的運作

- ▣ 計算機一定要能做算術。MIPS 組合語言的符號

`add a, b, c` # b 與 c 的和被置入 a 中

- 這種標記法非常嚴格，其規定了每一道 MIPS 算術指令只能執行一種運算且永遠一定使用三個變數
 - 這種語言每行最多包含一道指令
- ▣ 設計原則一：
簡單有利於規律性(simplicity favors regularity)
 - 在計算機設計中，簡單的設計通常更容易實現規律性和一致性，這有助於降低複雜性並提高效率。

2.3 計算機硬體的運算元

要被丟進 ALU 做運算的

- 算術指令的運算元有限制，必須是建構在硬體中數量有限的暫存器(registers)而來
 - 暫存器的數量有限，在MIPS架構中是32 個
- MIPS 架構中暫存器的大小是32 位元
 - 32 位元在MIPS 架構中被稱為字組(word)

- 設計原則二：愈小愈快(smaller is faster)
 - 很大數量的暫存器其內部的電子訊號必須傳遞更遠而需時更久，可能導致時脈週期時間增加
- 暫存器的有效運用對效能極為關鍵

使用暫存器來編譯C 基本敘述

將程式中的變數與暫存器關聯起來的工作是由編譯器負責。例如，

$$f=(g+h)-(i+j) ;$$

令變數f，g，h，i，j 分別存於暫存器\$s0，\$s1，\$s2，\$s3，\$s4 中

編譯出來的MIPS 碼為何？

使用暫存器來編譯C基本敘述

編譯出來的程式

add \$t0, \$s1, \$s2 # 暫存器\$t0 內含g+h

add \$t1, \$s3, \$s4 # 暫存器\$t1 內含i+j

sub \$s0, \$t0, \$t1 # f 得到\$t0-\$t1, 亦即(g+h)-(i+j)

記憶體運算元

- ▣ 複雜資料結構中的資料可能超過計算機中暫存器的數目，需存取於記憶體中
- ▣ MIPS必須具有能在暫存器及記憶體間轉移資料的指令
 - 該類指令稱為資料轉移指令(data transfer instructions)
— 載入 Load、儲存 Save 指令
- ▣ 將記憶體中的值複製至暫存器中的資料轉移指令為載入(load)
 - lw，表示載入字組(load word)

- ▣ 編譯器除了要將變數指定到各暫存器之外，還要將如同陣列及結構等資料結構配置(allocate)到記憶體的不同位置中，以位元組為定址的單位
- ▣ 在MIPS 中，字組的位址需由4 的倍數處開始
 - 這個規定稱為對齊限制(alignment restriction)

一個 Word 必定 4 個 byte!

■ 計算機的字組定址可分為兩種：

- 以最左邊(Big Endian)，或是最右邊 (Little Endian)的位元組位址作為字組的位址
- Little Endian: least-significant byte at least address
最小權重放在最低位

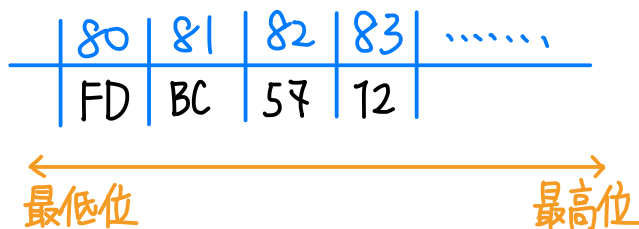
■ 相對於載入的指令傳統上稱為儲存(store)

- sw，表示儲存字組(store word)

Little Endian Sample:

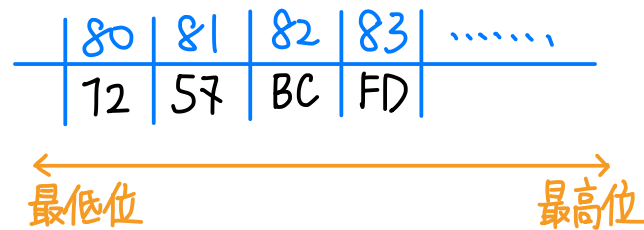
a value: 12 57 BC FD
 ← 權重大 權重小

mem:



Big Endian Sample:

mem:



Byte-Order (“Endianness”)

- For a multi-byte datum, which part goes in which byte?

→ 先將word 補到 8 的倍數 00 0F 42 40
Weight ↑ Weight ↓

- If \$1 contains 1,000,000 ($F4240_H$) and we store it into address 80:
- On a “big-endian” machine, the “big” end goes into address 80

		00	0F	42	40			
...	79	80	81	82	83	84	...	

- On a “little-endian” machine, it’s the other way around

		40	42	0F	00			
...	79	80	81	82	83	84	...	

← 低位 高位 →
14

Big-Endian vs. Little-Endian

(非intel陣營)

- Big-endian machines: MIPS, Sparc, 68000, PowerPC
 - MAC OS
- Little-endian machines: most Intel processors, Alpha, VAX, ARM
 - Windows, Linux
- No real reason one is better than the other...
- Compatibility problems transferring multi-byte data between big-endian and little-endian machines – **CAREFUL!**

- 許多程式具有比計算機中暫存器數目還要多的變數。
 - 編譯器通常將最常使用的變數存於暫存器而將其他置於記憶體
 - 以載入及儲存指令將變數在暫存器及記憶體間搬移
 - 資料若存在於暫存器而非記憶體，其存取較快速。並且一次可以存取多個暫存器輸出(register files有多個輸出及輸入)，獲得更高的效能及省能

常數或立即運算元

- ▣ 程式常需在運算中使用常數
 - 提供可具有一個常數運算元的算術指令
- ▣ 常數 0 可提供指令集有用的變化並而簡化設計
 - MIPS 將一個暫存器 \$zero 以線路將其值固定為0

2.4 有號及無號數字

▣ 以下圖示是MIPS 的字組中位元的編號方法以及 1011_2 的放置法：

MSB ↗ ↘ LSB

- 最不重要位元(least significant bit, LSB) 表示最右側的位元(下圖的位元0)，而最重要位元(most significant bit, MSB) 表示最左側的位元(位元31)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1

(32 位元寬)

■ 若運算的結果無法以所提供的有限的硬體位元表示，則稱之為溢位(overflow)

■ 可區分正負數的表示法 (都拿MSB當符號)(以8Bit為例: $\begin{matrix} 00000000 & 0 \\ 11111111 & 255 \end{matrix}$)

■ 符號大小(sign and magnitude)表示法

MSB 取補
 $\begin{matrix} 0000000 & (+1) & 00000000 & (+0) \\ 1000000 & (-1) & 10000000 & (-0) \end{matrix}$
↓ 斜 值 有兩零!

■ 有若干缺點 $(-127 \sim +127)$

■ 2的補數(two's complement)表示法：

負數取1 comp's 後+1
 $\begin{matrix} 0000000 & (+1) \\ 1111111 & (-1) \\ 0000000 & (+0) \end{matrix}$

■ 目前所有計算機都使用2 的補數表示法來表示有號數 $(-128 \sim +127)$
→ 2 Comp' 只有一種零

■ 1的補數(one's complement)

正數取反
表負數

$\begin{matrix} 0000000 & (+1) \\ 1111111 & (-1) \\ 0000000 & (+0) \\ 1111111 & (-0) \end{matrix}$ $(-127 \sim +127)$
> 1 comp' 有2個0

■ 偏移表示法(biased notation) 將一個數字偏移成該值加上偏移量(bias) 的表示法

0 ~ 255 中
↳ 128 當0
→ < 128 負數: $[127] - 1$ $[126] - 2$
→ > 128 正數: $[129] + 1$ $[130] + 2$

▣ Loads 以及算術指令均須分辨運算元是否為有號數

- 有號數載入指令需作為符號延伸(sign extension)
- sign extension

■ Ex: 8bit -> 16bit

- ^正 00001111 -> ^{前面補0} 00000000 00001111 (+15)
- ^負 11110001 -> ^{前面補1} 11111111 11110001 (-15)

▣ 載入位址不應為負

Example:

$lw \$S1, 20(\$S2) \rightarrow S1 \leftarrow M[S2+20]$

💡 Key: 一個CMD 32個Bit

這兩個加完不能是負的



→ 要先做位元延伸
才能放進ALU相加

2.5 在計算機中表示指令

將一道MIPS 指令轉換成機器指令

以符號形式表示的指令：

add \$t0,\$s1,\$s2

在真實MIPS 語言中的十進位以及二進位的表示法

名 稱	舉 例	註 解
32 個暫存器	\$s0-\$s7, \$t0-\$t9, \$zero, \$a0-\$a3, \$v0-\$v1, \$gp, \$fp, \$sp, \$ra, \$at	快速存取資料的地方。在 MIPS 中，資料必須要在暫存器裡才能執行運算，暫存器 \$zero 恆等於 0，而暫存器 \$at 則是保留給組譯器來處理值很大的常數。

每個 Reg 都有不同的編號! → 至少要5個 Bit 才能表示所有 Reg!

00000 ~ 11111
(0) (31)

十進位表示法：



第一以及最後一個欄位(上例中的0 以及32)合併起來告知MIPS 計算機該指令為加。

第二欄位指出加法運算的第一個來源運算元(source operand)的暫存器編號(17=`$s1`)

第三欄位指出另一個來源運算元(18=`$s2`)

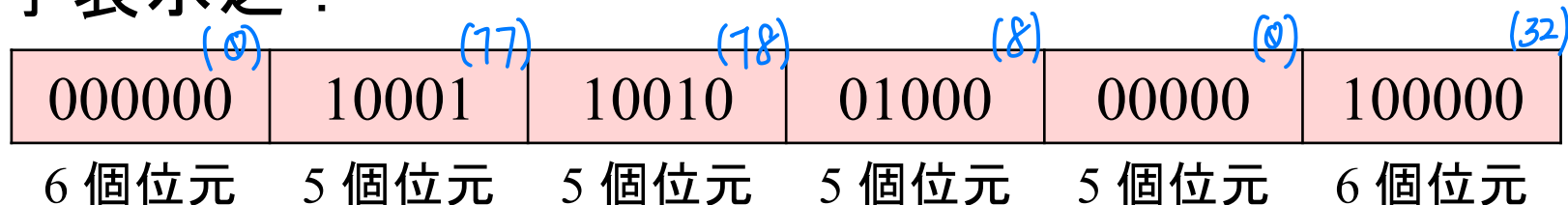
第四欄位指出目的暫存器的編號(8=`$t0`)

第五欄位在該指令中沒有用到，因此被給予0 這個值

該指令將暫存器`$s1` 及`$s2` 相加並將和置於暫存器`$t0` 中

將一道MIPS 指令轉換成機器指令

- 該指令相對於十進表示法，亦可將各欄位以二進數字表示之：



- 這種指令的擺置方式稱為指令格式(instruction format)。
 - 遵循 simplicity favors regularity 的設計原則，所有MIPS指令的長度均為32 個位元，並僅使用少數指令格式
 - 稱呼此種數字形式的指令為機器語言(machine language) (上為一種CMD格式範例，還有別種)
指令，一串該等指令為機器碼(machine code)

MIPS 欄位

$\text{add } \$S1, \$S2, \$S3 \Rightarrow \underset{(rd)}{S1} \leftarrow \underset{(rs)}{S2} + \underset{(rt)}{S3}$

■ 對MIPS 各欄位給予名稱： 有3个Reg的CMD $\Rightarrow$ R type CMD

op	rs	rt	rd	shamt	funct
----	----	----	----	-------	-------

6 個位元 5 個位元 5 個位元 5 個位元 5 個位元 6 個位元

- *op* : 該指令的基本運作，傳統上稱其為運作碼(opcode)
- *rs* : 第一個來源運算元暫存器
- *rt* : 第二個來源運算元暫存器
- *rd* : 目的運算元暫存器。其會得到運作的結果
- *shamt* : 位移量(shift amount)。Sll(邏輯左移)及srl(邏輯右移)指令使用 (指定要左/右移多少 Bit)
- *funct* : 功能。本欄位稱之為功能碼(function code)，配合特定op 欄位值組成算術及邏輯指令。

MIPS 欄位

■ 設計原則三：好的設計需要有好的折衷(good design demands good compromises)

- MIPS 設計者選擇的折衷是保持所有指令長度相同，因此對不同種類的指令有不同格式的需求
- 上述格式稱為R 型(指暫存器register)或R 格式

(有用到了3個Reg的)

■ 第二種指令格式稱為I 型(指立即值immediate)或I 格式，其用於立即值及數據轉移指令。I 格式的欄位為：

e.g. $\text{add } \$S1, \$S2, 30 \Rightarrow S1 \leftarrow S2 + 30$
 $\text{lw } \$S1, 20(\$S2) \Rightarrow S1 \leftarrow M[S2 + 20]$

I type 僅有2個Reg
↓

op	rs $S2$	rt $S1$	常數或位址 30
6 個位元	5 個位元	5 個位元	16 個位元

MIPS 欄位

- ▣ 第一個欄位的內容決定了所採用的格式：
 - 由此硬體即可得知如何解讀指令的後半

MIPS 機器語言

名 稱	型式	舉 例						註 解
加法	R	0	18	19	17	0	32	add \$s1,\$s2,\$s3
減法	R	0	18	19	17	0	34	sub \$s1,\$s2,\$s3
加立即值	I	8 <i>addi</i>	18	17		100		addi \$s1,\$s2,100
載入字組	I	35 <i>lw</i>	18	17		100		lw \$s1,100(\$s2)
儲存字組	I	43 <i>sw</i>	18	17		100		sw \$s1,100(\$s2)
欄位大小		6 位元	5 位元	5 位元	5 位元	5 位元	6 位元	所有 MIPS 指令的長度都是 32
6 欄 R 型式	R	op	rs	rt	rd	shamt	funct	算術指令型式
4 欄 I 型式	I	op	rs	rt	address			資料傳輸型式

圖2.6 到第2.5 節為止出現過的MIPS 架構。

R 和I 是到目前為止的二種MIPS 指令型式。這兩種指令格式前十六位元相同；兩者都包含 *op* 欄位，說明是哪一種運算；*rs* 欄位，指出一個來源運算元及 *rt* 欄位是另一個來源運算元，而在載入字組指令 *rt* 則是指目的運算元。R 型式將後十六位元分成 *rd* 欄位，指出目的暫存器；*shamt* 欄位會在第2.6 節解釋；*funct* 欄位指出 R 型式指令的運作。I 格式則將後十六位元當作 *address* 欄位

- ➡ 現今計算機均根據二個主要原則來建構：
 - 指令均以數字來代表
 - 程式儲存於可以被讀或寫的記憶體中，如同數字一般
- ➡ 此為內儲程式(stored-program)的概念；
 - 圖2.7 表示該概念；(P.28)
 - 具體而言，記憶體可存放編輯(editor)程式的原始碼、相對應的編譯過的(compiled)機器碼、該編譯過的程式所使用的文字(text)，甚至於產生該機器碼的編譯器



圖2.7 內儲程式的觀念

內儲程式讓計算機可以從執行會計程式在一瞬間轉換成幫助作家撰寫書籍。計算機之所以可以這樣轉換，僅需將記憶體載入不同的程式與資料，然後視需要告訴計算機從哪個位置開始執行程式。將指令與資料一視同仁大大簡化記憶體硬體和計算機系統的軟體。更仔細地講，資料所需要的記憶體技術也能使用在程式上，而且，例如編譯器程式能夠將使用方便人類閱讀的符號所寫成的程式，轉換成計算機所能了解的程式碼

2.6 邏輯運算

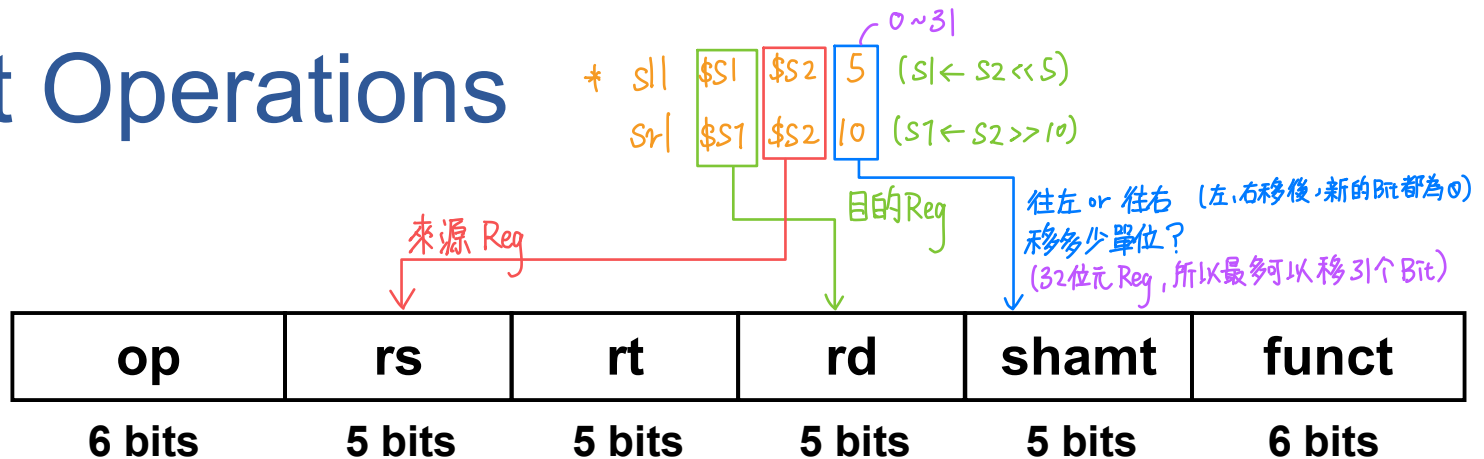
Operation	C	Java	MIPS
Shift left	<<	<<	sll
Shift right	>>	>>	srl
Bitwise AND	&	&	and, andi <i>Reg & Reg 做 AND 'Reg & 常數 做 AND</i>
Bitwise OR			or, ori <i>Reg & Reg 做 or 'Reg & 常數 做 or</i>
Bitwise NOT	~	~	nor <i>↘ MIPS 中沒有 not cmd</i>

圖2.8 C 和JAVA 的邏輯運算子及與之對應的MIPS 指令

MIPS 使用其中一個運算元為0 的NOR 來實現NOT

💡 Review NOR $[not(a \text{ or } 0)] = not\ a$
 $a \text{ nor } b = not(a \text{ or } b)$
 若其一為0, 則相當於0

Shift Operations



- shamt: how many positions to shift
- Shift left logical
 - Shift left and fill with 0 bits
 - **sll** by i bits multiplies by 2^i 左移 n 個 Bit 就是乘 2^n
- Shift right logical
 - Shift right and fill with 0 bits
 - **srl** by i bits divides by 2^i (unsigned only) 右移 n 個 Bit 就是除 2^n (無符號才算)

AND Operations

▣ Useful to **mask** bits in a word

- Select some bits, clear others to 0

遮住的地方不要变, 其他全部清0

and \$t0, \$t1, \$t2

$t0 \leftarrow t1 \& t2$

不变! (其他变0)

\$t2	0000 0000 0000 0000 0000 1101 1100 0000
\$t1	0000 0000 0000 0000 0011 1100 0000 0000
\$t0	0000 0000 0000 0000 0000 1100 0000 0000

OR Operations

- Useful to include bits in a word
 - Set some bits to 1, leave others unchanged

or \$t0, \$t1, \$t2

$t0 \leftarrow t1 \mid t2$

设成1 (其他变0)

\$t2	0000 0000 0000 0000 0000 1101 1100 0000
\$t1	0000 0000 0000 0000 0011 1100 0000 0000
\$t0	0000 0000 0000 0000 0011 1101 1100 0000

NOT Operations

- ▣ Useful to invert bits in a word
 - Change 0 to 1, and 1 to 0
- ▣ MIPS has NOR 3-operand instruction
 - $a \text{ NOR } b == \text{NOT} (a \text{ OR } b)$

`nor $t0, $t1, $zero` ←

**Register 0:
always read as
zero**

`$t1` 0000 0000 0000 0000 0011 1100 0000 0000

`$t0` 1111 1111 1111 1111 1100 0011 1111 1111

2.7 能作決定的指令(分支指令)

有這兩個cmd存在
— bne, beq ← 才能實現 if, goto。

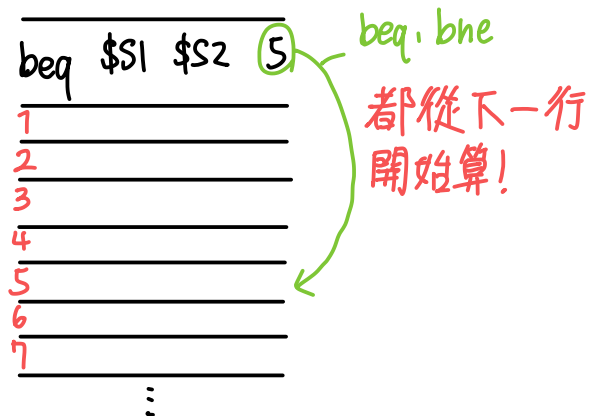
- MIPS 組合語言包含了兩個作決定的指令，二者均與 if 且含有 go to 的敘述類似

beq register 1, register 2, L1

Branch Equal: reg1, reg2 比較, 若相等往下跳 L1 個 Cmd

bne register 1, register 2, L1

Branch Not Equal



- 稱為條件式分支指令(conditional branches)

CPU 怎麼判斷相等?

— 法1: 相減。兩 Reg 減減看結果是不是 = 0。

— 法2: 互斥或。兩 Reg 做互斥或。= 0 就相等。

法2 e.g. $S1 \oplus S2$

S1 1011	S1 1011
S2 1011	S2 1101
0000	0110

在 MIPS 中比較兩 Reg 用法1

Compiling If Statements

■ C code:

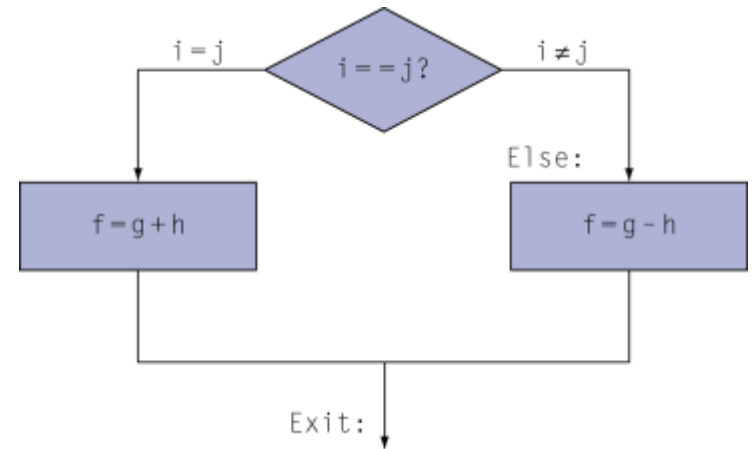
```
if (i==j) f = g+h;
else f = g-h;
```

- f, g, h, j, i in
\$s0, \$s1, \$s2, \$s3, \$s4...

■ Compiled MIPS code:

```
bne $s3, $s4, Else
add $s0, $s1, $s2 (f ← g+h)
j Exit
Else: sub $s0, $s1, $s2 (f ← g-h)
Exit: ...
```

Assembler calculates addresses



在组语中使用 label 表欲跳转行
而不用真正知道要跳去哪

Compiling Loop Statements 迴圈

■ C code:

32 bits! (每个都 4 byte)

```
while (save[i] == k) i += 1;
```

■ i in \$s3, k in \$s5, address of save in \$s6

■ Compiled MIPS code:

```

Loop: sll    $t1, $s3, 2    t1 = i * 4
      add    $t1, $t1, $s6
      lw     $t0, 0($t1)
      t0 = save[i]
      若 t0 != k, 跳走    bne    $t0, $s5, Exit
      若相等, 把 i + 1    addi   $s3, $s3, 1
      j      Loop
Exit:  ...
    
```

Handwritten notes:
 t1 = i * 4 + save 的位置 (t1 = &save[i])
 若 t0 != k, 跳走
 若相等, 把 i + 1 (\$s3)

\$S6 = &save
save 在記憶體中的位置

從 mem 拿出來給 t0

💡 lw CMD Review!
 lw \$t0, 0(\$t1)
 在記憶體中
 t0 ← M[t1+0]
 t1 的內容加 0

Memory

Addr.	Content
x-8	
x-4	
x	save[0] 4 Byte
x+4	save[1] 4 Byte
x+8	save[2] ...
x+12	save[3] ...
x+16	save[4] ...
	...

Handwritten notes:
 \$S6 →
 i=0
 i=1
 i=2
 i=3
 i=4
 x4 ⇒ << 2 (左移 2)!

More Conditional Operations

- ▣ Set result to 1 if a condition is true

- ▣ Otherwise, set to 0

- ▣ `slt rd, rs, rt` (`slt`: set less then) *rs & rt 比較*

- ▣ if ($rs < rt$) $rd = 1$; else $rd = 0$;

- ▣ `slti rt, rs, constant` *rs & 常數比較*

- ▣ if ($rs < \text{constant}$) $rt = 1$; else $rt = 0$;

- ▣ Use in combination with `beq`, `bne`

*$s1 < s2, t0 = 1$
 $s1 > s2, t0 = 0$*

```
slt $t0, $s1, $s2    # if ($s1 < $s2)
bne $t0, $zero, L     # branch to L
```

试试?

if ($i \leq j+k$) branch to L

$i = s0$

$j = s1$

$k = s2$

1	add \$t0, \$s1, \$s2
2	beq \$t0, \$s0, L
3	slt \$t1, \$s0, \$t0
4	bne \$t1, \$zero, L

$s2 < s1$



□ if ($\$s1 > \$s2$) branch to L ?

slt \$t0, \$s2, $<$ \$s1 # if ($\$s1 > \$s2$)

bne \$t0, \$zero, L # branch to L

$s2 \leq s1$

□ if ($\$s1 \geq \$s2$) branch to L ?

Beq \$s2, \$s1, L $\leftarrow$ 先看有沒有等於, 有等於就跳!

再比有沒有小於 $\rightarrow$ slt \$t0, \$s2, $<$ \$s1 # if ($\$s1 > \$s2$)

bne \$t0, \$zero, L # branch to L

□ if ($\$s1 \leq \$s2$) branch to L ?

Beq \$s1, \$s2, L $\leftarrow$ 先看有沒有等於, 有等於就跳!

slt \$t0, \$s1, $<$ \$s2 # if ($\$s1 < \$s2$)

bne \$t0, \$zero, L # branch to L

Branch Instruction Design

Branch Less than, Branch Greater Equal Than

- ▣ Why not `blt`, `bge`, etc?
- ▣ Hardware for $<$, $\geq$, ... slower than $=$, $\neq$
 - Combining with branch involves more work per instruction, requiring a slower clock → 分支CMD越多、越複雜, Clock (時脈) 越慢!
 - All instructions penalized!
- ▣ `beq` and `bne` are the common case ← 利用 `beq`、`bne` & `Slt`, 就能實現所有的條件判斷
- ▣ This is a good design compromise

Signed vs. Unsigned

- ▣ Signed comparison: `slt`, `slti`
- ▣ Unsigned comparison: `sltu`, `sltui`
- ▣ Example
 - `$s0 = 1111 1111 1111 1111 1111 1111 1111 1111` (-1)
 - `$s1 = 0000 0000 0000 0000 0000 0000 0000 0001`
 - `slt $t0, $s0, $s1 # signed`
 - $-1 < +1 \Rightarrow \$t0 = 1$
 - `sltu $t0, $s0, $s1 # unsigned`
 - $+4,294,967,295 > +1 \Rightarrow \$t0 = 0$

Case/Switch敘述

- 實現`switch` 最簡單的方式是經由一連串的條件測試，將該`switch` 敘述轉換為連續的`if-then-else` 敘述
- 在有些情況下，各種的可能處理方法其所對應各指令序列的位址可以有效率地被編排在一個所謂的跳躍位址表(jump address table)或跳躍表(jump table)中
 - 如此程式僅需索引該表即可跳到適當的指令序列
 - 為了支援該情況，MIPS 含有一暫存器跳躍(jump register, jr) 指令

Case/Switch敘述

switch a

```
{  
    case 1: sub1(); i+=1; break;  
    case 2: sub2(); i+=2; break;  
    case 5: sub3(); i+=3; break;  
    case 9: sub4(); i+=4; break;  
    case 10: sub5(); i+=5; break;  
}
```

Memory

Addr.	content
P	sub1(); i+=1;
U	sub2(); i+=2;
V	sub3(); i+=3;
W	sub4(); i+=4;
X	sub5(); i+=5;

jump address table

0	
1	P
2	U
3	
4	
5	V
6	
7	
8	
9	W
10	X

2.8 以計算機硬體來幫助程序呼叫

▣ 程式在執行一個程序時

在 call 一个 副程式時，
會先將其返回的記憶體位置記住！
(用 stack 維護)

1. 將參數置於程序可存取的位置
2. 將控制權移轉給程序 → 改動 Program Counter
3. 程序預先儲存會使用到的資源
4. 執行所需的工作
5. 將結果的值置於呼叫的程式可取得的地方
6. 還原程序先前預先儲存的資源，將控制交回原來呼叫的地方，原因是一個程序可以在程式中許多個點被呼叫

更多暫存器

$s0 \sim s7$ $t0 \sim t9$
都有任務! 暫存區

- 暫存器是計算機中可存放數據的最快的地方，儘可能多使用它們
 - MIPS 軟體在程序呼叫時對32個暫存器的配置遵循下列的慣例：
 - \$a0-\$a3：用以傳遞參數的4個參數(argument)暫存器
 - \$v0-\$v1：用以回傳值的2個值(value)暫存器
 - \$ra：用以回到程式呼叫點的返回位址(return address)暫存器

更多暫存器

- MIPS 也納入了一道專為程序呼叫的指令：其跳躍至一個位址並同時將下一道指令的位址保留於暫存器\$ra中
 - 跳躍並鏈結(jump-and-link, jal)指令的寫法是：
jal ProcedureAddress
 - 儲存於暫存器\$ra(暫存器31)中的”記憶體位置”稱為返回位址(return address)
 - 支援返回的情況：
 - 暫存器跳躍(jump register, jr)指令：
jr \$ra

非條件式跳躍	跳躍	j 2500	前往 10000	跳至目的位址
	透過暫存器跳躍	jr \$ra	前往 \$ra	用於 switch 敘述、程序反回
	跳躍並連結	jal 2500	\$ra=PC+4 前往 10000	用於程序呼叫

更多暫存器

還回去要恢復原狀

- ▣ 程序完成其任務後，所有呼叫者會用到的暫存器均需恢復其內含的值為程序被叫喚前的值
 - 理想資料結構是堆疊(stack)
 - MIPS 軟體預留第29 個暫存器(SP)作堆疊指標
(記憶體中)

更多暫存器

- 為了避免保留及恢復了某個不會被用到的暫存器
 - \$t0-\$t9：10 個在程序呼叫時不會受被呼叫者 (callee) 保存的暫存器 (副程式若使用了 t0~t9 的 Reg, 不会保留!)
 - \$s0-\$s7：8 個在程序呼叫時一定會受到保存的暫存器 (saved registers) (被呼叫者若使用它們，才會保留及恢復其值)

t0~t9 的值, 由呼叫者自己保存!

副程式若:

— 使用了 t0~t9, 隨使用, 不用還原, 主程式要想辦法記他原本的值, 到時候還回去主程式會自己想辦法還原!

— 使用了 s0~s7, 副程式一定要保存其值, 還回去時要把 s0~s7 還原

Leaf Procedure Example

■ C code:

```
int leaf_example (int g, h, i, j)
{ int f;
  f = (g + h) - (i + j);
  return f;
}
```

- Arguments g, h, i, j in \$a0, \$a1, \$a2, \$a3, f in \$s0 (hence, need to save \$s0 on stack)
- Result in \$v0

Leaf Procedure Example

■ MIPS code:

leaf_example:

放東西進 Stack (PUSH) (MOVE) 把東西拿出來 (POP)

```
addi $sp, $sp, -4  → 堆疊指標-4 (先挪一個位子出來放 $s0)
sw    $s0, 0($sp)  把 $s0 存到堆疊      Save $s0 on stack
add   $t0, $a0, $a1  t0 ← g + h      Procedure body
add   $t1, $a2, $a3  t1 ← i + j
sub   $s0, $t0, $t1  s0(f) ← t0 - t1
add   $v0, $s0, $zero  v0 ← s0 + 0    Result (MIPS 中沒有搬移CMD)
lw    $s0, 0($sp)    → 把堆疊最上面的東西傳給 $s0 (還原 $s0 的值)      Restore $s0
addi  $sp, $sp, 4    ↪ 堆疊指標還原
jr    $ra            ↪ 呼叫副程式返回位址      Return
```

巢狀的程序

- ▣ 如同我們在程序中使用暫存器時要謹慎，當呼叫非末端程序時，尤需更為謹慎

↳ 副程式還有呼叫副程式

- 使用到參數值時 (傳參的 $a0 \sim a3$ 是共用的)
- 返回位址的保存 (回傳結果的 $v0 \sim v1$ 也是共用的)

$\$a0-\$a3$ ：用以傳遞參數的4 個參數(argument)暫存器

$\$v0-\$v1$ ：用以回傳值的2 個值(value)暫存器

巢狀的程序

- 一種解決方法是將所有相關的**需要被保存的暫存器**均**推入堆疊**中：

- **呼叫者**負責將任何呼叫完成後仍需使用到的參數暫存器(\$a0-\$a3)及暫時暫存器(\$t0-\$t9)推入堆疊

— 讀 a0~a3, t0~t9
— 讀 \$ra, s0~s7
\$ra

- **被呼叫者**負責將**返回位址**及任何其會使用到的被保存的暫存器(saved registers, \$s0-\$s7)推入堆疊

- 圖2.11 歸納了跨越程序呼叫時會被被呼叫者保存的事物

巢狀呼叫時

被保存的暫存器：\$s0-\$s7	暫時暫存器：\$t0-\$t9
堆疊指標暫存器：\$sp	參數暫存器：\$a0-\$a3
返回位址暫存器：\$ra	(返回值)值暫存器：\$v0-\$v1

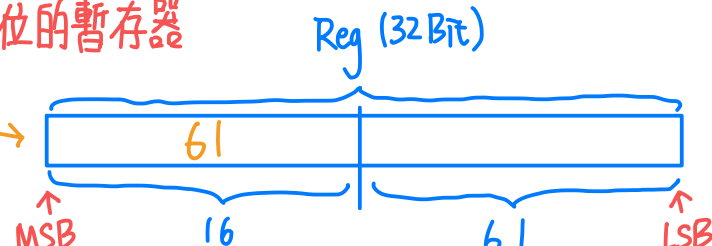
被呼叫者保存

呼叫者保存

2.9 MIPS 對32 位元立即值及位址的定址法

▣ 32 位元的立即值運算元

- 載入高位立即值(load upper immediate, lui)指令

- ex: lui \$s0, 61
16 Bit
- 將"立即值"載入高位的暫存器
- 

ex: 可將下方32 位元常數載入\$s0 的MIPS 組合碼為何？

0000 0000 0011 1101 | 0000 1001 0000 0000

61_{DEC} 2304_{DEC}

載入32 位元的常數

1. 使用lui 來將高位的16 個位元，即 61_{10} ，載入：

lui \$s0, 61 # $61_{10}=0000\ 0000\ 0011\ 1101$

暫存器\$s0的值是：

0000 0000 0011 1101 | 0000 0000 0000 0000

高 位

低 位

填值(或運算)

2. 置入低位的16 個位元，即 2304_{10} ：

ori \$s0, \$s0, 2304 # $2304_{10}=0000\ 1001\ 0000\ 0000$

暫存器\$s0 中最後的值為：

0000 0000 0011 1101 0000 1001 0000 0000

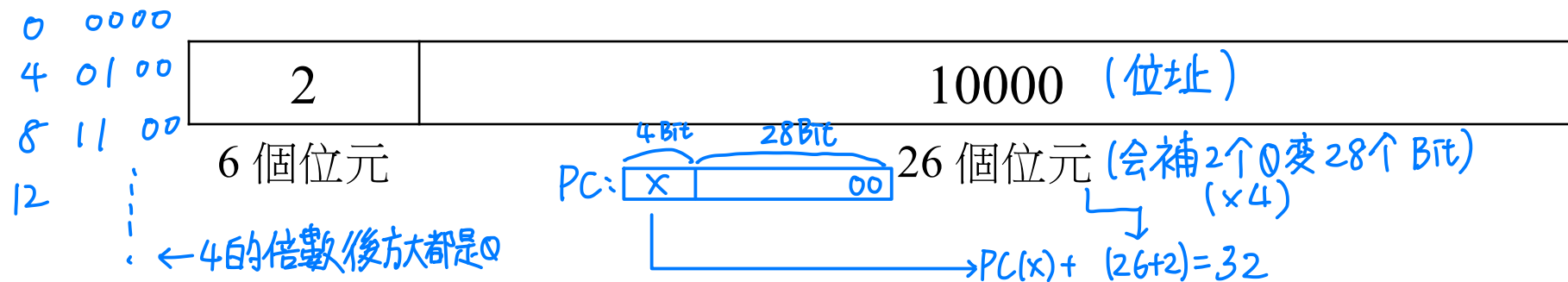
分支及跳躍的定址

R Type CMD : add \$S1 \$S2 \$S3 , sll , srl



I Type CMD : addi , lw , beq , bne ... ← 有常數項的都是 I Type CMD

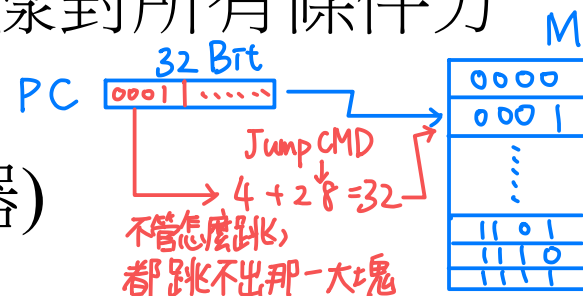
- 使用MIPS 的最後一個稱之為J(jump)型式的指令格式



- 由於條件分支的目的位址很可能相當靠近分支指令，MIPS 也和大多數新近的計算機一樣對所有條件分支均採用PC-相對定址法

- PC : program counter(程式計數器)

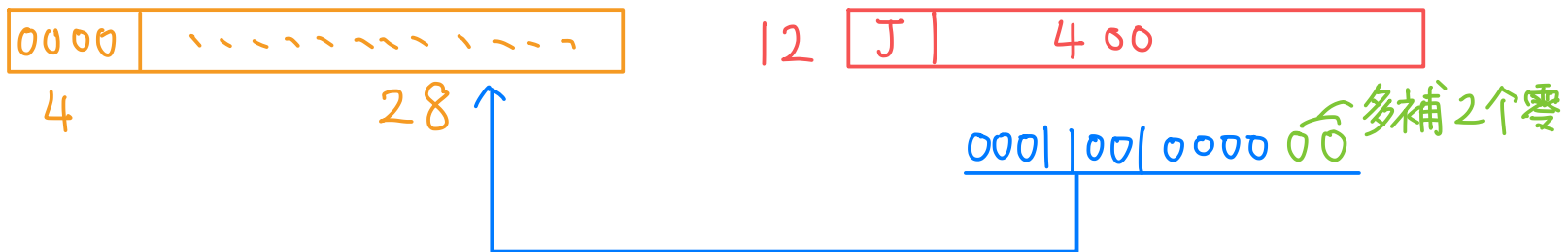
- 跳躍並聯結(jump-and-link)指令呼叫的程序卻不見得會在近處 (用位置表示) (jal)



分支及跳躍的定址

- MIPS 架構對跳躍(j)以及跳躍並聯結指令(jal)使用了J(jump)型式的格式來提供程序呼叫長的位址。
- 在PC-相對定址法中位址所代表的是距離目的指令的字組數目
- 跳躍指令中的26 位元位址欄代表的也是字組數
- MIPS 的跳躍指令將PC(program counter) 的低位 28 個位元取代掉並且保留其高位的4 個位元

PC:



MIPS 定址模式總結

- 圖2.16表示了每種定址模式如何決定其運算元。
- MIPS 具有下列定址模式：
 - ① 立即值定址法(immediate addressing)，其運算元為指令中所含的一個常數 e.g. `addi $S1 $S2 100` ($S1 \leftarrow S2 + 100$)
 - ② 暫存器定址法(register addressing)，其運算元為一個暫存器(中所含的值) e.g. `add $S1 $S2 $S3` ($S1 \leftarrow S2 + S3$)
 - ③ 基底或位移定址法(base or displacement addressing)，其運算元在位址為一個暫存器加上指令中的常數的記憶體位置中 e.g. `lw $S1, 100($S2)`
($S1 \leftarrow \underbrace{M[S2 + 100]}_{\text{in mem}}$)

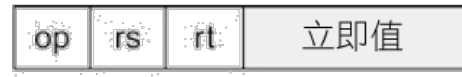
MIPS 定址模式總結

- ④ PC 相對定址法(PC-relative addressing)，其分支位址為PC 加上指令中的常數 e.g. `beq $S1 $S2 100`
- ⑤ 虛擬直接定址法(pseudodirect addressing)，其跳躍位址為指令中的26 個位元(譯者按：該26 位元表字組位址)再於其前串接上PC的高位位元

圖2.16 五種MIPS 定址模式的圖例

運算元為灰色的陰影部分，模式3 的運算元在記憶體裡，而模式2 的運算元在暫存器中。注意載入和儲存指令有存取位元組、半字組和字組的版本。模式1中，運算元為指令本身的其中16 個位元。模式4 和模式5 定址在記憶體裡的指令，模式4 將16 位元的位址向左位移2 位再加上PC，模式5 則是向左位移2位的26 位元位址串接PC 高位元的4 個位元

1. 立即定址法

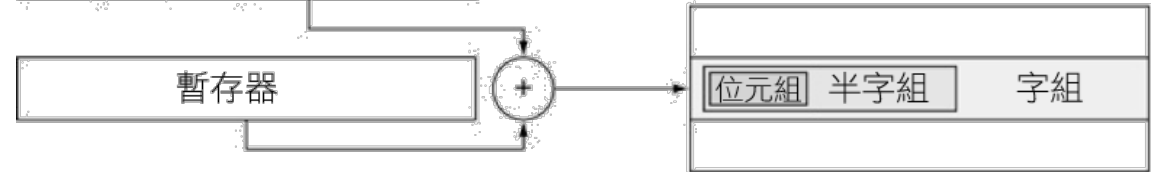


2. 暫存器定址法

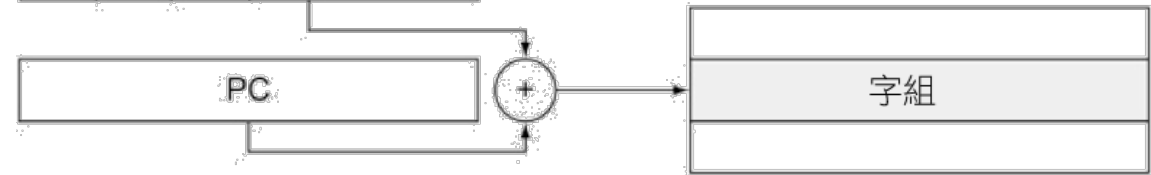


暫存器
暫存器

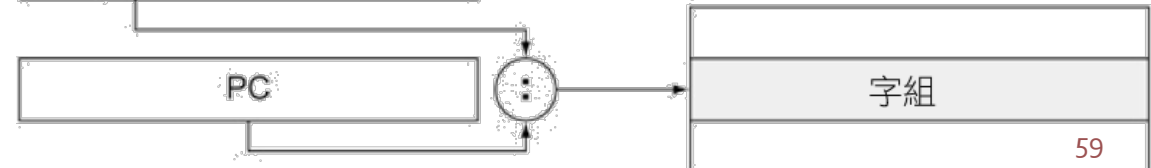
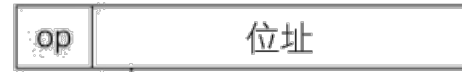
3. 基底定址法



4. PC 相對定址法



5. 虛擬直接定址法



解碼機器語言

I、R、J Type 第一个欄位是op

■ 圖2.17 顯示MIPS 機器語言各欄位的編碼

op(31:26)								
28-26	0(000)	1(001)	2(010)	3(011)	4(100)	5(101)	6(110)	7(111)
31-29								
0(000)	R 格式	Bltz/gez	jump	jump & link	branch eq	branch ne	blez	bgtz
1(001)	add immediate	addiu	set less than imm.	set less than imm. unsigned	andi	ori	xori	load upper immediate
2(010)	TLB	FlPt						
3(011)								
4(100)	load byte	load half	lwl	load word	load byte unsigned	load half unsigned	lwr	
5(101)	store byte	store half	swl	store word			swr	
6(110)	load linked word	lwcl						
7(111)	store cond. word	swcl						

op(31:26)=010000 (TLB), rs(25:21)								
23-21	0(000)	1(001)	2(010)	3(011)	4(100)	5(101)	6(110)	7(111)
25-24								
0(00)	mfc0		cfc0		mtc0		ctc0	
1(01)								
2(10)								
3(11)								

op(31:26)=000000 (R-format), funct(5:0)								
2-0	0(000)	1(001)	2(010)	3(011)	4(100)	5(101)	6(110)	7(111)
5-3								
0(000)	shift left logical		shift right logical	sra	sllv		sriv	srav
1(001)	jump register	jalc			syscall	break		
2(010)	mfhi	mthi	mflo	mtlo				
3(011)	mult	multu	div	divu				
4(100)	add	addu	subtract	subu	and	or	xor	not or (nor)
5(101)			set l.t.	set l.t. unsigned				
6(110)								
7(111)								

圖2.17 MIPS 指令的編碼。

這張符號表藉由行和列來表示欄位的數值。例如，在圖的最上面表示load word 列的數字為4 (指令的31-29位元為100₂)和行的數字為3(指令的28-26 位元為011₂)，所以對應的op 欄位(31-26 位元)值為100011₂。下面劃線表示該欄位用在其他地方。例如，0 列0 行(op=000000₂)的R 格式定義在圖的最下方。因此，在圖最下方4 列2 行的subtract 表示指令的funct 欄位(位元5-0)是100010₂ 以及op 欄位(位元31-26)是000000₂。圖最上方2 例1 行的floating point 值定義在第三章的圖3.18 中。Bltz/gez 是附錄A 中4 個指令的運算碼：bltz、bgez、bltzal 和bgezal。本章會說明以全名並且灰色來表示的指令，第三章會說明以助憶名稱及灰色來表示的指令。附錄A 說明所有的指令。

解碼機器語言

▣ 圖2.18 表示了所有的MIPS 指令格式。

名稱	欄 位						註 解
欄位大小	6 位元	5 位元	5 位元	5 位元	5 位元	6 位元	所有 MIPS 指令的長度都是 32 位元
R 格式	op	rs	rt	rd	shamt	funct	算術指令格式
I 格式	op	rs	rt	位址／立即值			傳輸、分支、imm 格式
J 格式	op	目標位址					跳躍指令格式

圖2.18 MIPS 指令格式

2.10 平行性與指令：同步 (x 不上, 跳 2-11)

- ▣ 鎖上(lock)及開鎖(unlock)兩種同步動作
 - 可直接地用於建立只允許單一處理器進入及運算的區域此種現象稱為互斥(mutual exclusion)
 - 用於建構更為複雜的同步機制
- ▣ 所需的關鍵能力是一組能夠對一個記憶體位址不可切分地(atomically)讀取以及修改的硬體低階動作(稱為primitives)
 - 如果缺乏硬體支援，建構該等基本同步低階動作的成本將極高，且隨處理器數目而增加

2.10 平行性與指令：同步

- 一個典型運作是不可切分的交換(**atomic exchange** 或 **atomic swap**)，其會互換暫存器及記憶體中兩個值的位置
 - 假設一個簡單**lock**，當其值為0 時表示其可被使用而為1 時表示其已被使用
 - 處理器欲設定該**lock** 值成1(即取用之)時，會嘗試以一個預設在暫存器中的1 來與代表該**lock** 的記憶體位址作交換
 - 交換回來的值若為1，表示已有其他處理器取得存取的權限；
 - 否則該值為0，**lock** 也已被改為1，可防止任何由其他處理器為了爭奪**lock** 而發出的交換指令取得0

2.10 平行性與指令：同步

- 另一種可能性則為使用一對指令，且讓第二道指令執行後會回傳一個值以顯示該指令對是否以沒有被切分的方式執行完成
 - 在MIPS 中這樣的一對指令包括載入一聯結(load linked) 條件儲存(store conditional)
 - 該二指令會連著來用：若該load linked 指令所指的記憶體位址之內容在load linked 之後以及store conditional 寫入該位址前被更動，則store conditional 的執行失敗
 - store conditional 被設計成不但會將一個暫存器的值存入記憶體，且會將該暫存器的值依執行的成功與否而設為1或0

- 雖然不可切分的交換在這裡是用於多處理器的同步，其亦可用於作業系統在一個處理器中處理多個程序(processes)時
 - ▶ 在一個處理器中為了確保沒發生(程序間的)互相干擾，在該對指令間若處理器作了程序切換(context switch)，則條件儲存也會失敗(見第五章)
- 其可被用來建構其他種類的同步低階動作，例如不可切分的比較與對調(atomic compare and swap)或不可切分的取得及遞增(atomic fetch-and-increment)
 - ▶ 這些個低階動作需要在ll 及sc 間用到額外的指令

2.11 翻譯與啟動一個程式

- ▣ 將一個C 程式轉換成可於計算機上執行的程式的四個步驟
 - 編譯(Compile) (Compile High Level Language to Assembly)
 - 組譯(Assemble) (Assembly to machine code by assembler)
 - 連結(Link) (obj file Link to .exe)
 - 載入(Load) (load to mem.)
- ▣ 詳細說明於編譯器與系統程式課程

編譯器(Compiler)與組譯器(Assembler)

- 編譯器將C 程式轉換成以符號形式來表示的組合語言程式
- 組譯器將組合語言程式翻成機器碼
- 組譯器亦可將指令常用的各種變形視為有效的指令
 - 硬體並不需要直接執行這些指令；讓它們能用於組合語言中以簡化翻譯以及程式撰寫
 - 這種指令稱為虛擬指令(pseudo instructions)

組譯器的虛擬指令(pseudo instructions)

- 例：MIPS 架構中並不存在的(虛擬)指令：

`move $t0,$t1` # 暫存器\$t0 得到暫存器\$t1(的值)

組譯器將這個組合語言指令轉換成相當於下方指令的機器語言指令：

`add $t0,$zero,$t1` # 暫存器\$t0 得到0+暫存器\$t1(的值)

- 其他虛擬指令例子：

`blt`(小於則分支)(轉換成`slt` 和`bne` 這兩道指令)

`bgt`(大於)、`bge`(大於等於)及`ble`(小於等於)。

立即值指令有16位元立即值的限制，MIPS組譯器卻允許將32位元的立即值載入暫存器(的組合指令)。

組譯器的虛擬指令(pseudo instructions)

- ❑ 虛擬指令提供MIPS 較硬體所直接製作者更為豐富的組合語言指令集。唯一的代價是保留一個給組譯器使用的暫存器\$at。
- ❑ 當 I 類指令載入超過 16 位的常數時，組譯器會需要運用 \$at 作為暫存區將大常數拆開，重新組合入暫存器中。

組譯器

- ▣ 組譯器也能接受各種基底的數字
- ▣ 組譯器將組合語言程式轉換成目標檔(object file)
 - 包含機器語言指令、數據以及將指令恰當置入記憶體中所需的資訊
 - 組譯器用符號表(symbol table)來追蹤分支及數據移動指令中所用到的標籤

聯結器 *Linker*

- ▣ 聯結編輯器(link editor)或聯結器(linker) 將所有相關而各自組譯成的機器語言程式「縫合」在一起
- ▣ 聯結器要執行的步驟有三個：
 - ① 將程式碼及數據的各模組(以符號代表的方式)配置於記憶體中。(此時並未真正置入記憶體，因仍有待解的問題如下，且尋找記憶體位置並置入是載入器loader 的責任。)
 - ② 決定出數據與指令所使用的標籤應代表的位址值
 - ③ 補正各個模組對內、對外參考的(位址或定址)參數值

聯結器

- 聯結器藉由每一個目標模組的符號表來釐清所有未被定義的標籤
- 當聯結器將一個模組安置入記憶體中，其所有的參考位址都必須重新定位來反映其真實位置
- ▣ 聯結器產生的是一個可在計算機上執行的可執行檔 (executable file)
(还不能執行)

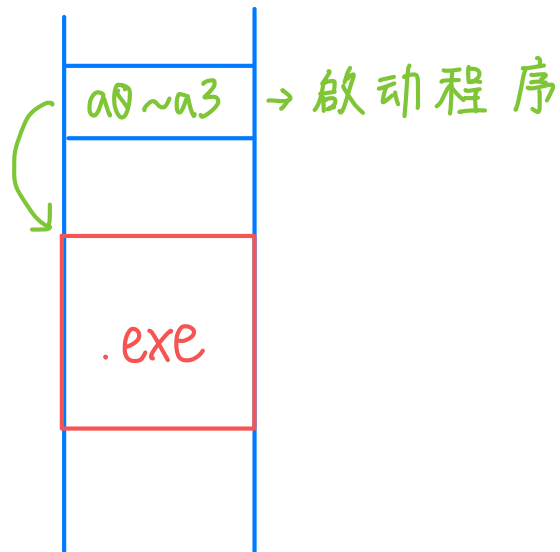
載入器

- ▣ 聯結成功後可產生執行檔，作業系統的載入器(loader)要將它讀到記憶體中然後開始執行
- ▣ 在UNIX 系統中載入器(loader)執行以下步驟：
 1. 讀取可執行檔的檔頭以獲知文字部分及數據部分的大小
 2. (在記憶體中)取得一塊足以容納文字及數據的位址空間 (連續的mem!)
 3. 將指令及資料複製入記憶體中
 4. 若有要傳遞給主程式的參數則複製到堆疊上

載入器

5. 初始化各機器暫存器並將堆疊指標設定到第一個可用的位置處
6. 跳躍到一個會複製參數到參數暫存器中並呼叫程式主程序的啟動程序。在主程序返回後，該啟動程序以一個exit系統呼叫來結束程式

$a0 \sim a3$



動態聯結的函式庫 (.dll)

- 傳統上，靜態的在程式執行前聯結各函式庫程序的作法雖然是呼叫函式庫最快的方法，然而其存在幾個缺失：
 - 函式庫程序成為可執行碼的一部分。若有新的修正過錯誤或支援新硬體的函式庫發表了，靜態聯結的程式仍然會使用到當初的舊版本
 - 該法會將可執行檔中所有呼叫到的函式庫程序，即使實際上沒有執行到的，全都載入。函式庫相對於程式可能很大；例如標準的C 函式庫大小是2.5 MB

動態聯結的函式庫

- 動態聯結的函式庫(dynamically linked libraries, DLLs)的設計等到程式執行時函式庫的程序才會被聯結以及載入
 - 所謂動態連結，就是把一些經常會共享的程式碼（靜態連結的OBJ程式庫）製作成DLL檔，當執行檔呼叫到DLL檔內的函式時，作業系統才會把DLL檔載入記憶體內，DLL檔本身的結構就是可執行檔，當程式有需求時函式才進行連結。透過動態連結方式，記憶體浪費的情形將可大幅降低。靜態連結函式庫則是直接連結到執行檔。

2.12 一個結合所有觀念的C 排序程式例

▣ 程序swap

($v[k] \Leftrightarrow v[k+1]$)

```
void swap(int v[], int k)
{
    int temp;
    temp = v[k];
    v[k] = v[k+1];
    v[k+1] = temp;
}
```

圖2.21 將記憶體裡兩個位置的內容做交換的C 程序

本節會在排序的例題中使用這個程序

程序swap

▣ 一般的步驟：

- ① 將暫存器配置給程式中的變數
- ② 產出程序主體的碼
- ③ 在程式呼叫時保存暫存器

swap 的暫存器配置

- MIPS 對參數傳遞的慣例是使用暫存器\$a0、\$a1、\$a2 以及\$a3
 - swap 僅有v 及k 兩個參數，因此它們會置於\$a0 及\$a1 中
- 另一個變數是temp，由於swap 是一個末端程序，因此我們用\$t0 來放置它

swap 程序本體的碼

$a0 = v[]$, $a1 = k$
 $t0 = temp$

```
sll $t1, $a1, 2    # 暫存器 $t1 = k*4
add $t1, $a0, $t1   # 暫存器 $t1 = v + (k*4)
                    # 暫存器 $t1 含有 v[k] 的位址
```

```
(temp = v[k])
lw $t0, 0($t1)      # 暫存器 $t0 (亦即 temp) = v[k]
lw $t2, 4($t1)      # 暫存器 $t2 = v[k+1]
                    # 指到 v 中的下一個元素
                    # (t2 = v[k+1])+4
```

```
                    v[k+1] 的 value    v[k] 的位址
sw $t2, 0($t1)      # v[k] = 暫存器 $t2
                    v[k] 的 value    v[k+1] 的位址
sw $t0, 4($t1)      # v[k+1] = 暫存器 $t0 (亦即 temp)
```

完整的swap 程序

程序本體

swap: sll \$t1, \$a1, 2	# 暫存器 \$t1 = k * 4
add \$t1, \$a0, \$t1	# 暫存器 \$t1 = v + (k * 4)
	# 暫存器 \$t1 含有 v[k]的位址
lw \$t0, 0(\$t1)	# 暫存器 \$t0 (temp) = v[k]
lw \$t2, 4(\$t1)	# 暫存器 \$t2=v[k+1]
	# 指到 v 的下一個元素
sw \$t2, 0(\$t1)	# v[k] = 暫存器 \$t2
sw \$t0, 4(\$t1)	# v[k+1] = 暫存器 \$t0 (temp)

程序返回

jr \$ra

返回呼叫此程序的程序

圖2.22 在圖2.21 中程序swap 的MIPS 組合碼

程序sort

- 以泡泡(bubble)或交換(exchange)排序法來對一個整數數列作排序。

```
void sort (int v[], int n)  
{  
    int i, j;  
    for (i = 0; i < n; i += 1) {  
        for (j = i - 1; j >= 0 && v[j] > v[j + 1]; j--=1) {  
            swap(v, j);  
        }  
    }  
}
```

圖2.23 對陣列v 進行排序的C 程序

sort 的暫存器配置

- ▣ 兩個參數v 及n 置於參數暫存器\$a0 及\$a1 中
- ▣ 指定暫存器\$s0 及\$s1 來分別存放i 及j

sort 程序本體的碼

- 本體包含兩個巢狀 *for* 迴圈和一個含有參數的對 *swap* 的呼叫。
- 首先對第一個 *for* 迴圈作翻譯：

```
for(i=0; i<n; i+=1){
```

- 第一個 *for* 迴圈碼的結構體即為：

sort 程序本體的碼

(addi \$s0, \$zero, 0)
move \$s0, \$zero

i=0

for1tst:slt \$t0, \$s0, \$a1

若 $s0 \geq a1$ ($i \geq n$)

則暫存器 $t0=0$

beq \$t0, \$zero, exit1 # 若 $s0 \geq a1$ ($i \geq n$) 則

前往 exit1

.....

(第一個 for 迴圈的本體)

.....

addi \$s0, \$s0, 1

i+=1

j for1tst

跳躍至外部迴圈測試處

exit1:

sort 程序本體的碼

- ▣ 第二個 *for* 迴圈在 C 中的形式是：

```
for (j=i-1; j>=0 && v[j]>v[j+1]; j-=1){
```

- ▣ 第二個 *for* 迴圈的結構體成為：

sort 程序本體的碼

$j \geq 0$ 才做
↓
用 $j < 0$ 判斷

$v[j] > v[j+1]$?
→ $v[j+1] \leq v[j]$?
等於不見了?
→ 等於, 兩個調換也
沒關係, 所以還是
會執行交換, 但不影響結果

相等跳走 exit2
不相等, 繼續執行!
→ 做 swap!

```
addi $s1, $s0, -1    # j=i-1
for2tst: slt $t0, $s1, 0    # 若 $s1<0(j<0)則暫存器 $t0=1
                                0      j
bne $t0, $zero, exit2    # 若 $s1<0(j<0)則前往 exit2

                                j
sll $t1, $s1, 2    # 暫存器 $t1=j*4
                                t2=8*v[j]
add $t2, $a0, $t1    # 暫存器 $t2=v+(j*4)
lw $t3, 0($t2)    t3=v[j]    # 暫存器 $t3=v[j]
lw $t4, 4($t2)    t4=v[j+1]    # 暫存器 $t4=v[j+1]
                                v[j+1]  v[j]
slt $t0, $t4, $t3    # 若 $t4≥$t3 則暫存器 $t0=0
                                v[j+1]  v[j]
                                ← beq $t0, $zero, exit2 # 若 $t4≥$t3 則前往 exit2

.....
(第二個 for 迴圈的本體)
.....

addi $s1, $s1, -1    # j--=1
                                j for2tst
                                檢查是否符合條件
exit2:
```

sort 中的程序呼叫

- ▣ 第二個 *for* 迴圈中的本體：

`swap(v, j);`

- ▣ `jal swap` 跳到 `swap` 執行
把 `PC+4` 存到 `$ra`

在sort 中傳遞參數

- **sort** 程序需要將兩個數字放在暫存器\$**a0** 及 \$**a1** 中，然而**swap** 程序也需要將其二個參數置於同樣的兩個暫存器中
- 一種解決的辦法是，首先將 \$**a0** 及 \$**a1** 複製至\$**s2** 及 \$**s3** 中：

```
move $s2, $a0v # 複製參數 $a0 至 $s2 中  
move $s3, $a1n # 複製參數 $a1 至 $s3 中
```

然後以這兩道指令將參數傳遞給**swap**：

```
move $a0, $s2 # swap 的第一個參數是 v  
move $a1, $s1 # swap 的第二個參數是 j
```

保存sort 中的暫存器

- 由於sort 本身即是一個會被人呼叫的程序
 - 我們必須將其返回位址存於\$ra
- sort 程序也會用到\$s0、\$s1、\$s2 和\$s3 這些被保存的暫存器
 - 它們也必需被保存起來
- 於是sort 程序的開始部分即為

sort 會需要用 \$ra
保存返回地址

但sort 中的 swap
也會用到 \$ra

保存sort 中的暫存器

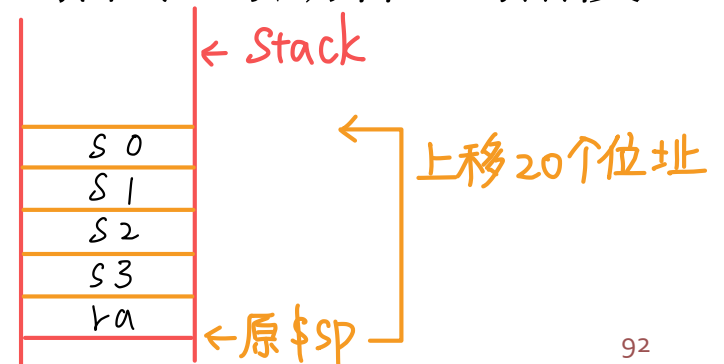
sort 本自己的返回位址也要存起來!

```
addi $sp,$sp,-20 # 在堆疊上騰出 5 個暫存器的空間
SW   $ra,16($sp) # 保存 $ra 於堆疊上
SW   $s3,12($sp) # 保存 $s3 於堆疊上
SW   $s2, 8($sp) # 保存 $s2 於堆疊上
SW   $s1, 4($sp) # 保存 $s1 於堆疊上
SW   $s0, 0($sp) # 保存 $s0 於堆疊上
```

$s0 \sim s7$
→ 被呼叫者保存
 $t0 \sim t9$
→ 呼叫者保存

sort 本身也是副程式, sort 內會用到 $s0 \sim s3$, 所以开始前也要先保存其值!

- 程序的結尾處只不過逆轉這些指令的動作，然後再加上 `jr` 以返回



完整的sort 程序

保存暫存器的值

sort:	addi \$sp, \$sp, -20	# 在堆疊裡挪出空間給五個暫存器
	sw \$ra, 16(\$sp)	# 將 \$ra 保存在堆疊裡
	sw \$s3, 12(\$sp)	# 將 \$s3 保存在堆疊裡
	sw \$s2, 8(\$sp)	# 將 \$s2 保存在堆疊裡
	sw \$s1, 4(\$sp)	# 將 \$s1 保存在堆疊裡
	sw \$s0, 0(\$sp)	# 將 \$s0 保存在堆疊裡

圖2.24 圖2.23 中程序sort 的MIPS 組合碼

程序本體

搬移 參數	move \$s2, \$a0 move \$s3, \$a1	# 將參數 \$a0 複製到 \$s2(保存 \$a0) # 將參數 \$a1 複製到 \$s3(保存 \$a1)
外部 迴圈	move \$s0, \$zero for1tst: slt \$t0, \$s0, \$s3 beq \$t0, \$zero, exit1	# i = 0 # 若 \$s0 $\geq$ \$s3(i $\geq$ n)則暫存器 \$t0 = 0 # 若 \$s0 $\geq$ \$s3(i $\geq$ n)則分支到 exit1
內部 迴圈	addi \$s1, \$s0, -1 for2tst: slti \$t0, \$s1, 0 bne \$t0, \$zero, exit2 sll \$t1, \$s1, 2 add \$t2, \$s2, \$t1 lw \$t3, 0(\$t2) lw \$t4, 4(\$t2) slt \$t0, \$t4, \$t3 beq \$t0, \$zero, exit2	# j = i - 1 # 若 \$s1 < 0(j < 0)則暫存器 \$t0 = 1 # 若 \$s1 < 0(j < 0)則分支到 exit2 # 暫存器 \$t1 = j * 4 # 暫存器 \$t2 = v + (j * 4) # 暫存器 \$t3 = v[j] # 暫存器 \$t4 = v[j + 1] # 若 \$t4 $\geq$ \$t3 則暫存器 \$t0 = 0 # 若 \$t4 $\geq$ \$t3 則分支到 exit2
傳遞參 數然後 呼叫	move \$a0, \$s2 move \$a1, \$s1 jal swap	# swap 的第一個參數為 v(原來的 \$a0) # swap 的第二個參數為 j # swap 的程式碼在圖 2.25
內部 迴圈	addi \$s1, \$s1, -1 j for2tst	# j -= 1 # 跳到內層迴圈的測試
外部 迴圈	exit2: addi \$s0, \$s0, 1 j for1tst	# i += 1 # 跳到外部迴圈的測試

圖2.24 圖2.23 中程序sort 的MIPS 組合碼

恢復暫存器的值

exit1:	lw	\$s0, 0(\$sp)	# 由堆疊恢復 \$s0	) 还原 s0~s3 Reg
	lw	\$s1, 4(\$sp)	# 由堆疊恢復 \$s1	
	lw	\$s2, 8(\$sp)	# 由堆疊恢復 \$s2	
	lw	\$s3, 12(\$sp)	# 由堆疊恢復 \$s3	
	lw	\$ra, 16(\$sp)	# 由堆疊恢復 \$ra	→ 还原返回指標
	addi	\$sp, \$sp, 20	# 恢復堆疊指標	

程序返回

jr \$ra # 回到呼叫的程序

圖2.24 圖2.23 中程序sort 的MIPS 組合碼

| CH2 上到这

- 有一個最佳化方法稱為程序內嵌(procedure inlining)
- (程序)內嵌在本例中可省掉4 道指令

瞭解程式效能

gcc 最佳化	相對效能	時脈週期數 (百萬)	指令個數 (百萬)	CPI
沒有	1.00	158,615	114,938	1.38
O1(中等)	2.37	66,990	37,470	1.79
O2(全部)	2.38	66,521	39,993	1.66
O3(程序整合)	2.41	65,747	44,993	1.46

圖2.25 編譯器對泡泡排序進行最佳化之效能、指令個數和CPI 的比較

程式對包含100,000 個亂數字組的陣列作排序。該等程式執行於時脈速率為3.06GHz 的Pentium 4 以及533 MHz 的系統匯流排和2 GB 的PC2100 DDR 同步動態隨機存取記憶體(SDRAM)。使用的作業系統是Linux 2.4.20 版

瞭解程式效能

語言	執行方法	最佳化選項	Bubble Sort 相對效能	Quicksort 相對效能	Quicksort 相對於 Bubble Sort 的加速
C	編譯器	無	1.00	1.00	2468
	編譯器	01	2.37	1.50	1562
	編譯器	02	2.38	1.50	1555
	編譯器	03	2.41	1.91	1955

圖2.26 兩個以C 寫作的排序演算法經過最佳化編譯器處理後相對於其本身未經最佳化的效能

最後一行表示**Quicksort**較**Bubble Sort**所具有的效能優勢。這些程式都是在和圖2.25 中所用的相同系統上執行

2.13 進階教材：編譯C 語言

- ▣ 本節提供C 編譯器如何工作
- ▣ 瞭解目前的編譯器技術對於瞭解效能的意義極為重大
- ▣ 通常是一至二個學期的課程

2.14 實例：ARMv7(32 位元)指令

- ▣ 圖2.27 列出ARM 與MIPS兩者的相似處
 - 兩者主要的不同在於MIPS 的暫存器較多而ARM 的定址模式較多

	ARM	MIPS
發表日期	1985	1985
指令大小(位元數)	32	32
位址空間(大小、模式)	32 bits, 不分區	32 bits, 不分區
資料是否需對齊	需對齊	需對齊
數據定址模式	9	3
整數暫存器(數量、模式、大小)	15 個 32 位元通用暫存器	31 個 32 位元通用暫存器
I/O	記憶體對映	記憶體對映

圖2.27 ARM 與MIPS 指令集的相似處

定址模式

定址模式	ARM	MIPS
暫存器運算元	×	×
立即值運算元	×	×
暫存器值 + 偏移值(移位或具基底的)	×	×
暫存器值 + 暫存器值(具索引的)	×	—
暫存器值 + 縮放的暫存器值(縮放的)	×	—
暫存器值 + 偏移值並更新暫存器值	×	—
暫存器值 + 暫存器值並更新暫存器值	×	—
自動遞增、自動遞減	×	—
PC-相對的資料存取	×	—

圖2.29 數據的定址模式縱覽。

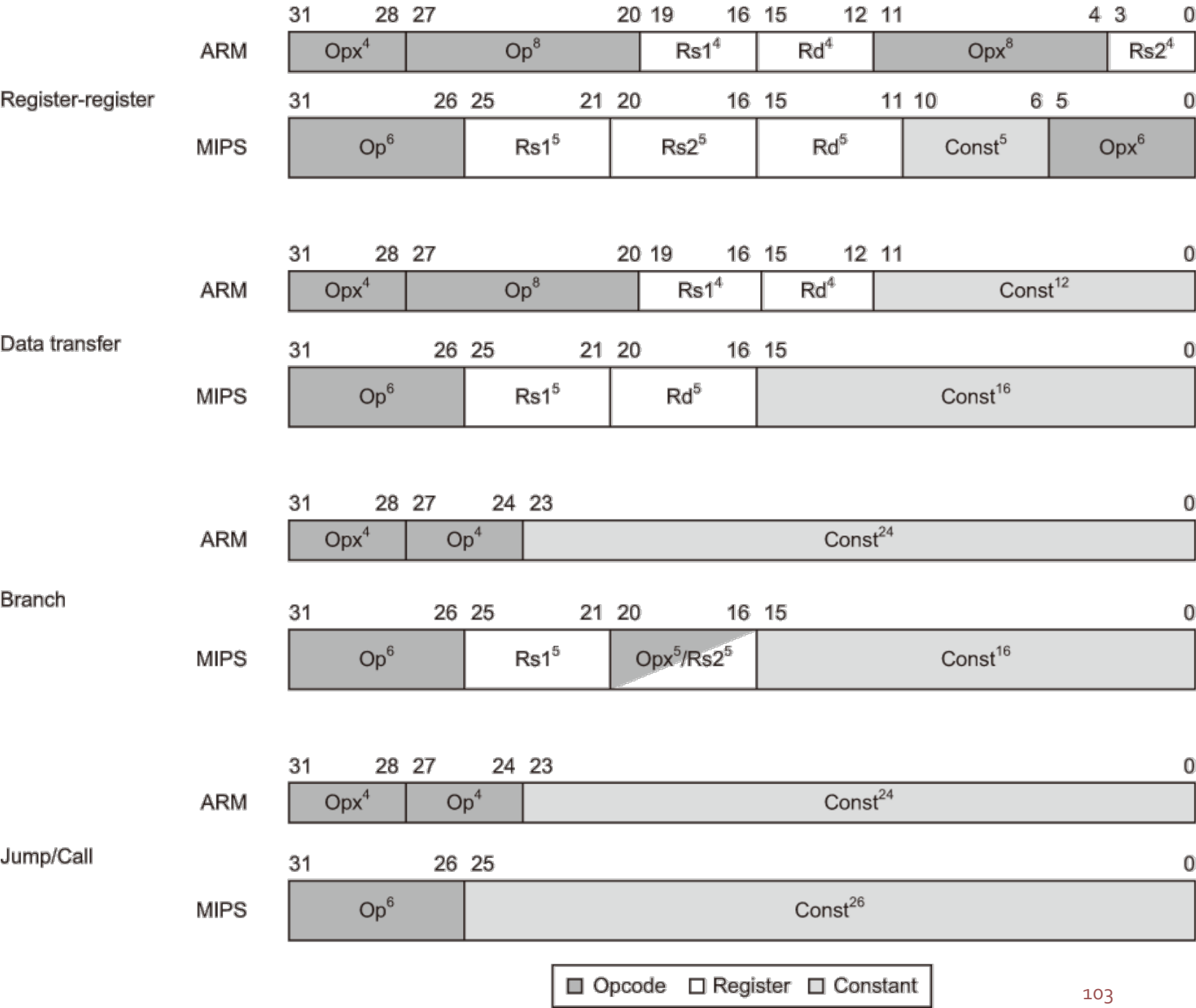
ARM 使用暫存器間接定址以及暫存器值+偏移值間接定址兩種不同的模式，而非僅以後者的偏移值中置入0來達成前者。**ARM** 在資料型態為半字組或字組時會將偏移值左移1或2位元以增加定址範圍

比較以及條件分支

- MIPS 以暫存器的內容來決定條件分支
- ARM 則使用傳統的四個存於程式現況字組 (program status word) 中的狀態碼位元 (condition code bits)：負號 (negative)、零 (zero)、進位 (carry) 和滿溢 (overflow)
 - ARM 的所有指令都可以根據狀態碼來作有條件的執行
- 圖2.30 說明ARM 與MIPS 的指令格式

圖2.30
ARM 與
MIPS 的
指令格式

差異來
自於該
架構有
16 個或
32 個暫
存器



ARM 獨有的特色

▣ 圖2.31 列出MIPS 中所沒有的幾道算術邏輯指令

名 稱	定 義	ARM v.4	MIPS
載入立即值	$Rd = Imm$	mov	addi \$0,
反相	$Rd = \sim(Rs1)$	mvn	nor \$0,
移動	$Rd = Rs1$	mov	or \$0,
向右旋轉	$Rd = Rs1 \gg i$ $Rd_{0 \dots i-1} = Rs1_{31-i \dots 31}$	ror	
反及	$Rd = Rs1 \& \sim(Rs2)$	bic	
反轉的減法	$Rd = Rs2 - Rs1$	rsb, rsc	
對多字組整數加法的支援	CarryOut, $Rd = Rd + Rs1 + OldCarryOut$	adcs	—
對多字組整數減法的支援	CarryOut, $Rd = Rd - Rs1 + OldCarryOut$	sbc	—

圖2.31 MIPS 中所沒有的ARM 算術/邏輯指令

ARM 獨有的特色

- ARM 還有可以保存一群暫存器、稱為區塊載入及儲存(block loads and stores)的指令
 - 一個指令就可以把16 個暫存器中的每一個選擇性地載入或儲存至記憶體中
 - 可以在程序進入或返回時保存或恢復暫存器
 - 也可用於大塊的記憶體複製

2.15 實例：x86 指令

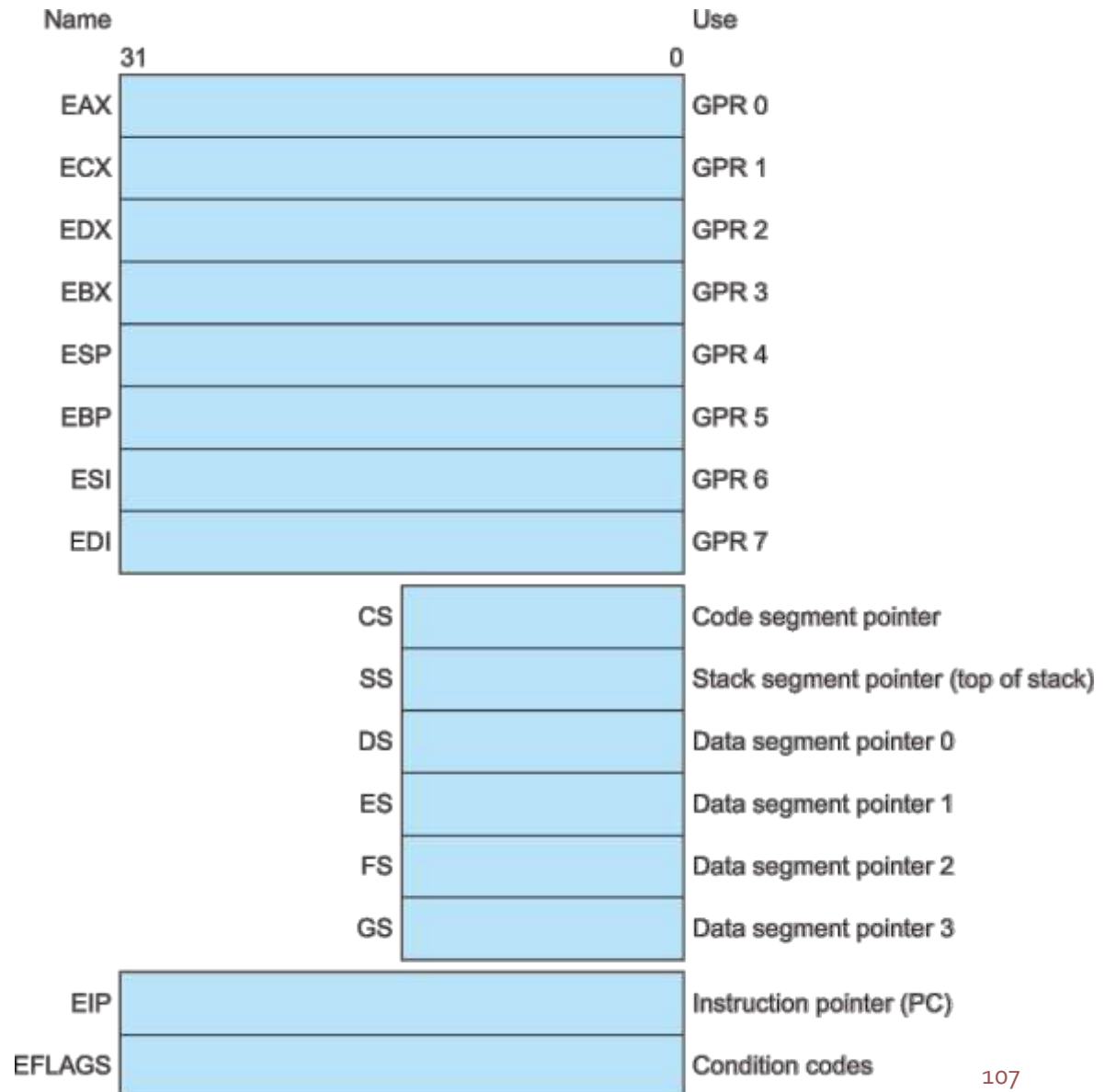
Intel x86 的演化

- ▣ x86 演化中的重要里程碑：見課本

x86 的暫存器以及數據定址模式

圖2.32 80386 的
暫存器集

從80386 開始，最上面的8 個暫存器擴充為32 位元，而且可以當作通用型暫存器使用



x86 的暫存器以及數據定址模式

來源 / 目的運算元型態	第二個來源運算元
暫存器	暫存器
暫存器	立即值
暫存器	記憶體
記憶體	暫存器
記憶體	立即值

圖2.33 算術、邏輯和數據傳輸指令的指令型態

x86 允許上述的運算元組合。唯一的限制是不可以有記憶體－記憶體模式。立即值的長度可能是**8**、**16** 或是**32** 位元；暫存器可以是圖2.32 裡**14** 個主要暫存器中的任何一個(除了**EIP** 或是**EFLAGS**)

x86 的暫存器以及數據定址模式

模 式	說 明	暫存器的限制	MIPS 的等效指令碼
暫存器間接定址	位址在暫存器裡	不可以是 ESP 或是 EBP	lw \$s0,0(\$s1)
使用 8 位元或是 32 位元位移量的基底模式	位址為基底暫存器的內容加上位移量	不可以是 ESP	lw \$s0,100(\$s1) # 位移量大小 ≤ 16 位元
基底加可縮放的索引	位址為基底 + ($2^{\text{scale}} \times$ 索引) Scale 的值為 0、1、2 或是 3	基底：任何 GPR 索引：不可以是 ESP	mul \$t0,\$s2,4 add \$t0,\$t0,\$s1 lw \$s0,0(\$t0)
基底加可縮放的索引以及 8 位元或 32 位元的位移量	位址為基底 + ($2^{\text{scale}} \times$ 索引) + 位移量 Scale 的值為 0、1、2 或是 3	基底：任何 GPR 索引：不可以是 ESP	mul \$t0,\$s2,4 add \$t0,\$t0,\$s1 lw \$s0,100(\$t0) # 位移量大小 ≤ 16 位元

圖2.34 x86 32 位元定址模式與暫存器使用上的限制以及等效的MIPS 碼

使用在ARM 或MIPS 中所沒有的基底加可縮放的索引的定址模式是為了省卻要把暫存器中的索引轉換成位元組位址時的乘以4(縮放因素為2)(見圖2.25 及2.27)。縮放因素為1 用於16 位元的資料，為3 則用於64 位元的資料。縮放因素為0 表示位址不做縮放。如果在第二種以及第四種模式中的位移量需使用多於16 個位元，則MIPS 的等效碼中需要多兩道指令：一道lui 來載入位移量的高位16 位元以及一道add 將高位的16 位元位址與基底暫存器\$s1 相加(Intel 給予所謂的基底定址模式兩個不同的名稱——基底的或索引的——然而它們基本上是相同的，我們在這裡一併以基底定址稱之)

x86 的整數運作

- 8086 支援 8 位元(位元組)以及16 位元(字組)兩種數據型態
- 80386 在 x86 中加上了 32 位元的位址以及數據(稱為雙字組double words)
- x86 的整數運作可被劃分成四個主要類別：
 - ① 數據搬移指令，包含move、push及pop
 - ② 算術以及邏輯指令，包含test、整數及十進數的算術運算
 - ③ 控制流程(control flow)，包含條件分支、無條件跳躍、呼叫及返回
 - ④ 串(string)指令，包含串搬移及串比較

x86 的整數運作

指 令	功 能
je 名稱	若相等(狀態碼){EIP=名稱}; EIP-128 <= 名稱 < EIP+128
jmp 名稱	EIP=名稱
call 名稱	SP=SP-4;M[SP]=EIP+5; EIP=名稱
movw EBX, [EDI+45]	EBX=M[EDI+45]
push EST	SP=SP-4;M[SP]=ESI
pop EDI	EDI=M[SP];SP=SP+4
add EAX, #6765	EAX=EAX+6765
test EDX, #42	用 EDX 和 42 設定狀態碼(旗標)
movsl	M[EDI]=M[ESI];EDI=EDI+4; ESI=ESI+4

圖2.35 一些典型的x86 指令和其功能

在圖2.36 列出常出現的運作。CALL 會將下一個指令的EIP 儲存在堆疊裡。(Intel 的EIP 就是PC)

x86 指令的編碼

指 令	意 義
控制	條件和無條件分支
jnz、jz	如果條件成立則跳躍至 $EIP + 8$ 位元偏移量；JNE(表 JNZ)，JE(表 JZ)為另一種名稱
jmp	無條件跳躍－8 位元或是 16 位元偏移量
call	副程式呼叫－16 位元偏移量；將返回位址推入堆疊裡
ret	從堆疊裡 pop 出返回位址並且跳到位址所在的地方
loop	迴圈分支遞減 ECX；如果 $ECX \neq 0$ 則跳躍到 $EIP + 8$ 位元的差量
數據傳輸	在暫存器之間或是在暫存器和記憶體之間搬移數據
mov	在兩個暫存器或是暫存器和記憶體之間搬移
push、pop	將來源運算元推入堆疊裡；將運算元從堆疊的頂端移除後放入暫存器
les	從記憶體載入到 ES 和其中一個 GPRs

圖2.36 x86 上一些典型的運算

許多運作使用暫存器－記憶體格式，其來源或是目的運算元之一可以是記憶體位置，另一則可以是暫存器或是立即值

x86 指令的編碼

指 令	意 義
算術、邏輯	使用數據暫存器和記憶體體的算術和邏輯運算
add、sub	將來源運算元加到目的運算元；將來源運算元從目的運算元減去；暫存器－記憶體格式
cmp	比較來源和目的運算元，暫存器－記憶體格式
shl、shr、rcr	向左移位；邏輯的向右移位；與進位狀態位元一起向右旋轉
cbw	將 EAX 最右邊 8 個位元的位元組轉換成 EAX 右邊的 16 位元字組
test	來源和目的運算元的邏輯 AND 以設定狀態碼
inc、dec	遞增目的數值，遞減目的數值
or、xor	邏輯 OR；互斥 OR；暫存器－記憶體格式
字串	在字串運算元之間搬移；長度由一個重複次數的前置標示來表示
movs	藉由遞增 ESI 和 EDI 來將字串從來源運算元複製到目的運算元；可能會重複動作
lods	將字串中的一個位元組、字組或是雙字組載入到 EAX 暫存器

圖2.36 x86 上一些典型的運算

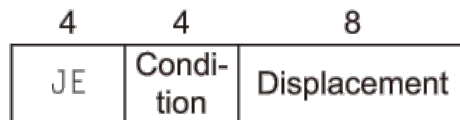
許多運作使用暫存器－記憶體格式，其來源或是目的運算元之一可以是記憶體位置，另一則可以是暫存器或是立即值

x86 指令的編碼

- ▣ 80386 的指令編碼非常複雜
 - 使用了許多不同的指令格式
 - 指令長度從沒有運算元的一個位元組到最長的15個位元組
- ▣ 部分指令的指令格式

圖2.37 典型的x86 指令格式

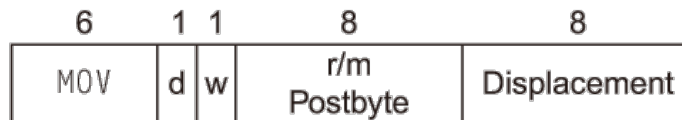
a. JE EIP + displacement



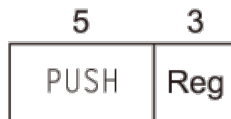
b. CALL



c. MOV EBX, [EDI + 45]



d. PUSH ESI



e. ADD EAX, #6765



f. TEST EDX, #42



圖2.37 典型的x86 指令格式

圖2.38 說明後置位元組的編碼。許多指令包含1 位元的w 欄位來說明運算是位元組或是雙字組(double word)。在MOV 裡的d 欄位是用在可能會將資料搬進/出記憶體指令來說明搬移的方向。**ADD** 需要32 位元的立即值欄位，因為在32 位元的模式裡，立即值不是8 位元就是32 位元。在**TEST** 裡的立即值欄位長度為32 位元，因為在32 位元的模式裡，沒有8 位元的立即值測試。整體來說，指令長度在1 到17 位元組之間。長度會比較長是來自於額外的1 位元組前置碼、使用4 個位元組的立即值和4 個位元組的位址位移值、使用2 位元組的運作碼、以及使用一個位元組的可縮放索引模式的標示碼(scaled index mode specifier)

x86 的結論

- ▣ x86難以設計製造
- ▣ 幸好的是x86 架構中最常被用到的部分在實作上並不太困難
 - Intel 一直很快地提昇整數程式的效能
 - 編譯器也一定要避免使用到架構中難以製作成快速線路的部分

2.16 實例：ARMv8(64 位元)指令

- ▣ 在2013 年面世
 - ARM 做了一個全面的大翻修
- ▣ 首先，相較於MIPS，ARM捨棄了幾乎所有v7 中不尋常的特性：
 - (指令中)不再具有條件式執行的欄位，而v7中幾乎每道指令都有這個欄位
 - 立即值欄位就是一個12 位元的常數，而非如v7 中基本上是對某函數的輸入以間接產出常數
 - ARM 捨棄了Load Multiple 與Store Multiple 指令
 - PC (程式計數器)不再是一般用途的暫存器之一，以避免寫入時不慎造成非預期的分支動作

2.16 實例：ARMv8(64 位元)指令

- 其次，ARM 加入了原來所沒有的在MIPS 中的有用特性：
 - V8 擁有編譯器寫作者一定會喜歡的32 個一般用途暫存器。如同MIPS，有一個暫存器是以接線固定成內容為0，雖然在load 與store 指令中它的索引值又用以代表堆疊指標
 - 它的定址模式對ARMv8 中所有字組大小均適用，而在ARMv7中並非如此
 - 它納入ARMv7 中沒有的divide 指令
 - 它納入MIPS 中branch if equal 及branch if not equal 的等效指令

2.17 謬誤與陷阱

- ▣ 謬誤：更強大的指令帶來更高的效能
- ▣ 謬誤：以組合語言撰寫程式以得最高效能
 - 編譯器與組合語言程式師間的戰爭的情況是人類正節節敗退
 - 即使手寫可以產出更快的程式碼，我們仍需考慮手寫組合語言碼可能帶來的顧慮：
 - 更長的撰寫及除錯時間，喪失了碼的可攜性以及維護這種碼的困難度

2.17 謬誤與陷阱

- 謬誤：在商業用途上二進碼相容性的重要程度意謂著成功的指令集不會再作改變
- 陷阱：忘記了在位元組定址的機器中連續字組的地址相差並非是1
- 陷阱：在一個自動變數(`automatic variable`)所被賦予定義的程序之外以一個指標(`pointer`)來指向它

2.18 總結評論

- ▣ 內儲程式(stored-program)計算機的兩個原則
- ▣ 有三個設計原則可以引導指令集設計者如何獲得一個精緻的平衡：
 1. 規則性易導致簡單的設計
 2. 愈小就愈快
 3. 好的設計需要有好的折衷
- ▣ 使經常的情形變快這個理念
- ▣ 可以允許透過使用硬體並無法執行的符號指令來「擴張」指令集
- ▣ 抽象化這個理念的

2.19 歷史觀點與進一步閱讀

- ▣ 回顧指令集架構(ISAs)的演進歷史
- ▣ 述及程式語言與編譯器的簡史
- ▣ 回顧高階語言計算機架構與精簡指令集計算機(reduced instruction set computer)架構間各種具爭議性的主題